

Clase 207 — Forense de memoria RAM con Volatility

Parte: **9 — Forense digital y respuesta a incidentes** · Fuente: *Ligh, Case, Levy, Walters — The Art of Memory Forensics* (Wiley, 2014) 🕒 Duración estimada: **140 min** · Nivel: **Avanzado**

🎯 Objetivo

Aprender a adquirir y analizar volcados de memoria RAM con Volatility 3 para descubrir lo que el disco no revela: procesos ocultos, conexiones de red activas, inyección de código, malware sin archivo y credenciales en memoria. Al terminar podrás cazar amenazas que solo viven en RAM.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Adquirir** volcados de memoria en Windows y Linux de forma forense.
2. **Enumerar** procesos, DLLs, handles y conexiones desde un volcado.
3. **Detectar** inyección de código y procesos ocultos.
4. **Extraer** ejecutables y artefactos de la memoria.
5. **Usar** Volatility 3 con sus plugins principales.

📖 Temas

#	Tema	Por qué importa
1	Por qué la RAM importa	Malware fileless vive ahí
2	Adquisición de memoria	Orden de volatilidad máximo
3	Enumeración de procesos	Base de todo análisis
4	Conexiones de red	C2 y exfiltración
5	Inyección de código	Malware avanzado
6	Extracción de binarios	Recuperar el malware
7	Detección de ocultamiento	Rootkits y DKOM
8	Volatility 3 y perfiles	La herramienta estándar

📖 Definiciones y características

- **Volcado de memoria:** copia del contenido de la RAM en un momento dado. Característica: es la evidencia más volátil; se pierde al apagar.
- **Malware fileless:** código que se ejecuta solo en memoria sin tocar disco. Característica: invisible para antivirus basados en archivos.

- **Inyección de código:** introducir código en el espacio de otro proceso. Característica: se detecta por regiones de memoria RWX inesperadas.
- **DKOM (Direct Kernel Object Manipulation):** técnica de rootkit que oculta procesos alterando estructuras del kernel. Característica: `pslist` no lo ve, `psscan` sí.
- **pslist vs psscan:** el primero recorre la lista enlazada de procesos; el segundo escanea memoria por firmas. Característica: la discrepancia delata ocultamiento.
- **malfind:** plugin que busca regiones de memoria sospechosas (RWX). Característica: detector clásico de inyección.
- **Perfil / símbolos:** metadatos del kernel para interpretar estructuras. Característica: Volatility 3 los descarga automáticamente.

Herramientas y preparación

- **Adquisición:** **FTK Imager** o **WinPmem** (Windows), **AVML** o **LiME** (Linux).
- **Análisis:** **Volatility 3** (Python 3). Instala con `pip install volatility3`.
- **Muestras:** usa un volcado de una VM propia o los volcados de práctica públicos (por ejemplo, imágenes de entrenamiento de Volatility). **Nunca ejecutes malware fuera de un laboratorio aislado y desechable.**

Laboratorio guiado

Adquiere memoria de una VM propia o usa una muestra de entrenamiento pública.

1. Adquiere la memoria (Windows, WinPmem):

```
powershell winpmem_mini.exe memoria.raw
```

En Linux con AVML:

```
bash ./avml memoria.lime
```

2. Lista procesos:

```
bash vol -f memoria.raw windows.pslist
```

3. Busca procesos ocultos comparando con psscan:

```
bash vol -f memoria.raw windows.psscan
```

Cualquier PID en `psscan` que no esté en `pslist` es sospechoso. 4. Revisa el árbol de procesos para relaciones padre-hijo raras:

```
bash vol -f memoria.raw windows.pstree
```

5. Enumera conexiones de red:

```
bash vol -f memoria.raw windows.netscan
```

6. Caza inyección de código:

```
bash vol -f memoria.raw windows.malfind
```

7. Vuelca un proceso sospechoso para análisis:

```
bash vol -f memoria.raw -o ./salida windows.dumpfiles --pid 1337
```

8. Revisa DLLs cargadas y líneas de comando:

```
bash vol -f memoria.raw windows.cmdline vol -f memoria.raw windows.dlllist --pid 1337
```

Ejercicios

1. Explica por qué se adquiere RAM antes que disco.
2. Detecta un proceso oculto comparando pslist y psscan.
3. Identifica una conexión de red sospechosa con netscan.
4. Usa malfind para hallar una región RWX inyectada.
5. Extrae el ejecutable de un proceso malicioso de la memoria.
6. Reconstruye la línea de comandos de un proceso con `cmdline`.

Reto verificable

A partir de un volcado de memoria (propio o de entrenamiento) que contenga un proceso malicioso inyectado, identifícalo, documenta cómo lo detectaste y extrae su código para análisis.

Criterio de aceptación: entregas (a) el PID y nombre del proceso malicioso, (b) la evidencia de inyección (salida de `malfind` con región RWX), (c) su conexión de red si la hay, y (d) el binario extraído con `dumpfiles`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Unsatisfied requirement / símbolos	Volatility no halló símbolos del kernel. Deja que los descargue o provee el ISF correcto.
pslist no ve el malware	Ocultamiento por DKOM. Usa <code>psscan</code> .
Volcado corrupto o truncado	Adquisición interrumpida. Repite con la máquina estable.
Plugins de Vol 2 no funcionan	Sintaxis distinta en Vol 3. Usa <code>windows.<plugin></code> .
netscan vacío	Volcado de un SO no soportado o muy antiguo. Verifica la versión.

Preguntas frecuentes

 **¿Volatility 2 o 3?** 3 es el estándar actual, en Python 3 y con descarga automática de símbolos. 2 sigue vivo por compatibilidad con plugins antiguos.

 **¿Cómo detecto malware fileless?** En memoria: procesos sin archivo en disco, inyección RWX (malfind), y PowerShell/WMI en `cmdline`. El disco no lo mostraría.

 **¿Puedo sacar contraseñas de la RAM?** A veces sí (hashes, tokens, incluso texto plano). Trátalas como datos sensibles y protégelas.

? **¿Por qué pslist y psscan difieren?** pslist confía en la lista del SO (manipulable); psscan escanea la memoria cruda por firmas y encuentra lo oculto.

Referencias

- Ligh, Case, Levy, Walters — *The Art of Memory Forensics*, Wiley 2014.
- Volatility Foundation: <https://www.volatilityfoundation.org/>
- Volatility 3 docs: <https://volatility3.readthedocs.io/>
- WinPmem / AVML: <https://github.com/Velocidex/WinPmem> · <https://github.com/microsoft/avml>

Siguiente clase

Clase 208 - Forense de red